

Le labo 7 n'a pas été implémenté, mais voici un rapport expliquant ce que j'aurais fait

Contexte & objectifs

- Passer d'une orchestration synchrone (HTTP) à une chorégraphie asynchrone via bus d'événements (RabbitMQ/Kafka).
- Event Sourcing : journaliser tous les événements métier, rejouables pour reconstruire l'état.
- CQRS : séparer Command (écritures) et Query (lectures) avec des projections.
- Saga chorégraphiée : coordination de la commande sans orchestrateur central (chaque service publie/réagit).
- Conserver l'observabilité (Prometheus/Grafana), le caching (Redis), la gateway et le load balancing.

Architecture — initiale (L5/L6) → finale (L7)

Avant (Lab 5 & 6)

API Gateway (Krakend/NGINX) → microservices FastAPI

Orchestrateur HTTP pour la commande

Redis cache (GET), DB relationnelle, Prometheus/Grafana

Après (Lab 7)

Event Bus (RabbitMQ/Kafka) entre services

Plus d'orchestrateur : Saga chorégraphiée (événements successifs + compensation)

Event Store (PostgreSQL table events) + Read Models (tables de projection)

CQRS : endpoints de lecture sur projections, commandes via events

ADR — décisions structurantes

ADR-01 — Passer au Pub/Sub (RabbitMQ/Kafka)

Pourquoi : découplage, résilience, extension facile (nouveaux consommateurs).

Impacts : latence parfois plus faible (batch/async), besoin d'idempotence & DLQ.

ADR-02 — Saga chorégraphiée (pas orchestrée)

Pourquoi : éviter point de défaillance unique, aligné microservices autonomes.

Impacts : logique distribuée + gestion d'échecs via événements compensatoires.

ADR-03 — Event Sourcing + CQRS

Pourquoi : auditabilité, relecture, vues de lecture optimisées.

Impacts : Event Store, projections, consistency eventual côté lecture.

Événements, topics et contrats (extraits)

Topics (Kafka) / Exchanges (RabbitMQ)

- order.events
- cart.events
- stock.events
- customer.events
- reporting.events

Contrats JSON (exemples)

```
// cart.events: CartItemAdded
{
  "eventId": "uuid",
  "type": "CartItemAdded",
  "timestamp": "2025-07-30T12:34:56Z",
  "payload": {
    "orderId": "o-123",
    "productId": 42,
    "qty": 2,
    "storeId": 1,
    "price": 19.99
  }
}
```

```
// stock.events: StockReserved | StockReservationFailed
{
  "eventId": "uuid",
  "type": "StockReserved",
  "timestamp": "2025-07-30T12:35:01Z",
  "payload": {
    "orderId": "o-123",
    "reservedLines": [{"productId": 42, "qty": 2, "storeId": 1}]
  }
}
```

```
// order.events: OrderCompleted (ou OrderCompensated)
{
  "eventId": "uuid",
  "type": "OrderCompleted",
  "timestamp": "2025-07-30T12:35:10Z",
  "payload": {
    "orderId": "o-123",
    "total": 39.98,
    "currency": "CAD"
  }
}
```

Event Store & Projections

Schéma Event Store (PostgreSQL)

- Table events(event_id PK, type, aggregate_id, aggregate_type, timestamp, payload_jsonb)
- Index: idx_events_agg_ts(aggregate_id, timestamp DESC)

Projections (Read Models)

- orders_read(order_id, status, total, updated_at, ...)
- stock_read(product_id, store_id, qty_avail, ...)
- Stratégie : consommateurs abonnés projetant les events vers ces tables.

CQRS — Endpoints

Command (écritures)

- POST /cart/{id}/items → publie CartItemAdded
- POST /cart/{id}/checkout → publie OrderReadyForCheckout

Query (lectures via read models)

- GET /orders/{id} → lit orders_read
- GET /stock/{storeId} → lit stock_read
- GET /reporting/dashboard → lit projections + cached (Redis)

Implémentation — comment (sur base existante)

Objectif : mettre à jour vos services existants sans tout réécrire.

Publier des événements là où vous faisiez des updates directs

- Ex: dans `cart_service.checkout_panier`, après validation → `publish("OrderReadyForCheckout", ...)`
- Ex: `stock_service.enregistrer_vente` → publie `StockReserved` ou `StockReservationFailed`

Consommer les événements pertinents

- `stock_service` consomme `CartItemAdded` → réserve le stock → publie `StockReserved|Failed`
- `reporting_service` consomme `OrderCompleted` → met à jour `orders_read` + `dashboards`

Event Store

Hook commun `EventPublisher` qui append chaque event en DB (events) puis push sur le bus

Idempotence

Clé `event_id` vérifiée côté consumer → ignorer doublons

Compensation (saga)

Si `StockReservationFailed`, `cart_service` publie `OrderCompensated` (ex: annule items / notifie client)

Observabilité

- Compteurs Prometheus : `events_published_total`, `events_consumed_total`, `sagas_success_total`, `sagas_failed_total`
- Dashboards Grafana pour topics (taux, latence émission→consommation)

Performances attendues & impact

- Moins de couplage → services tolèrent des pannes (file d'attente)
- Lecture rapide via projections + Redis (déjà en place côté L4/L5)
- Charge DB amortie (moins de writes synchrones côté requêtes web)
- Scaling ciblé : consommer plus vite un topic particulier

Limites & risques

- Complexité opérationnelle (broker, DLQ, replays)
- Débogage distribué (corrélation id / trace id recommandé)
- Cohérence éventuelle sur les vues de lecture
- Gestion stricte de l'idempotence et des timeouts

Plan de validation (sans tout recoder)

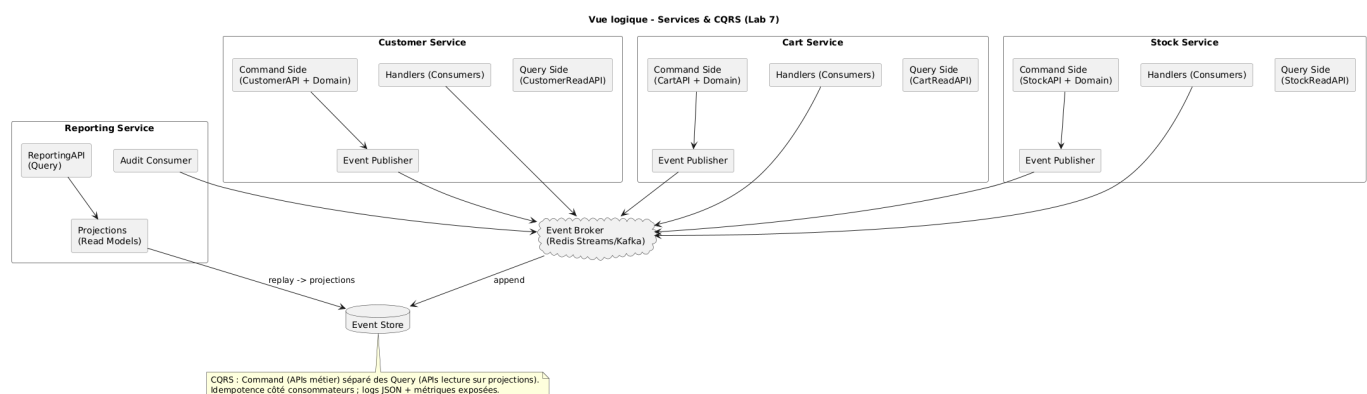
- Chemins heureux : panier → réservation stock → completion → projections OK
- Chemins d'échec : forcer StockReservationFailed → vérifier compensation et états (cart remis)
- Relecture : vider orders_read puis replay events (script) → vues reconstruites
- Métriques : vérifier dans Grafana (events/s, sagas OK/KO, latence émission→consommation)

Ce qui aurait été implémenté

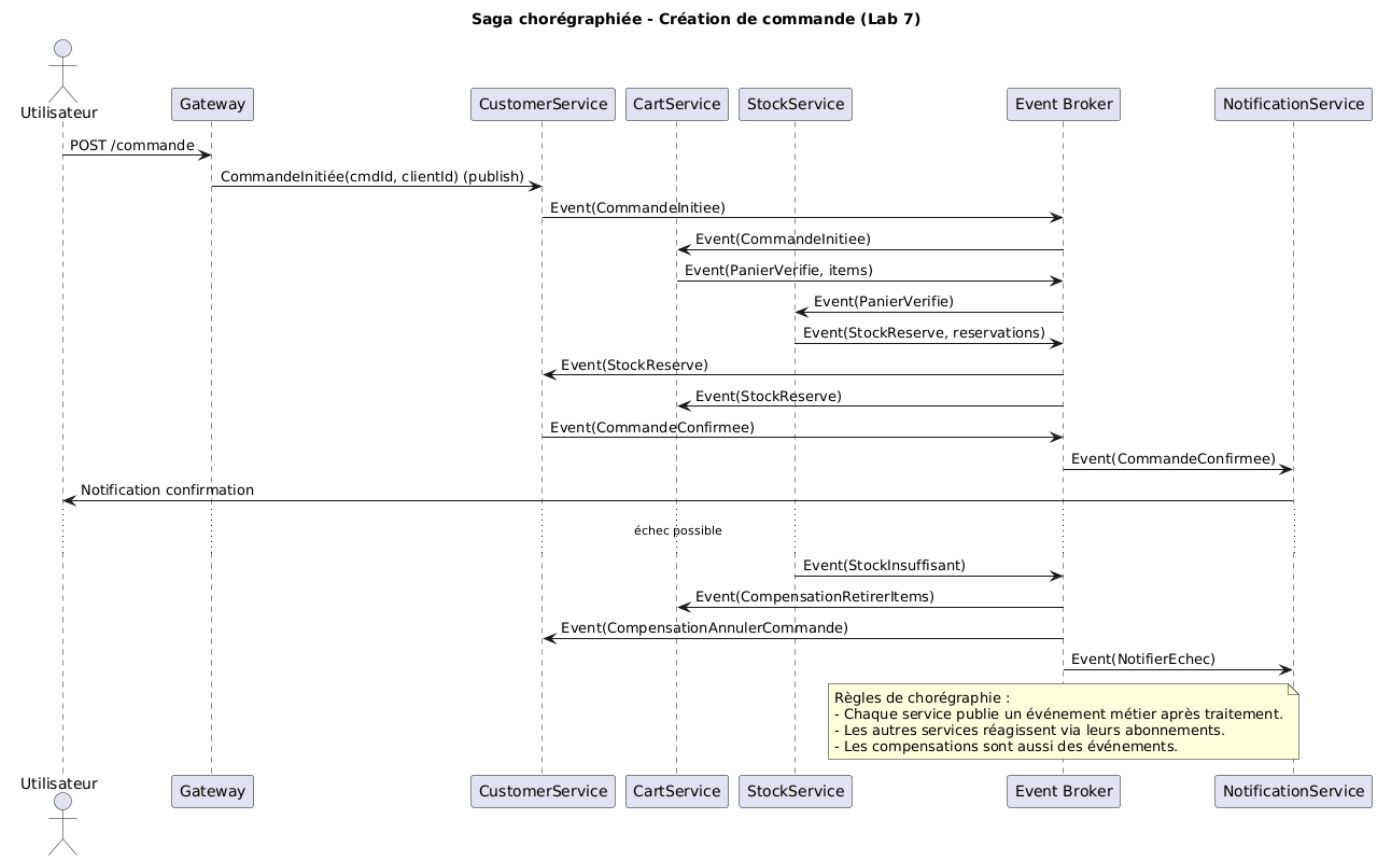
- Intégration Pub/Sub sur les services existants (publish & consume d'événements clés)
- Event Store PostgreSQL + projections pour lectures (CQRS)
- Saga chorégraphiée pour la commande (réservation stock → completion/compensation)
- Observabilité étendue : métriques d'événements et de saga
- Idempotence & replay supportés

Diagrammes

Vue Logique

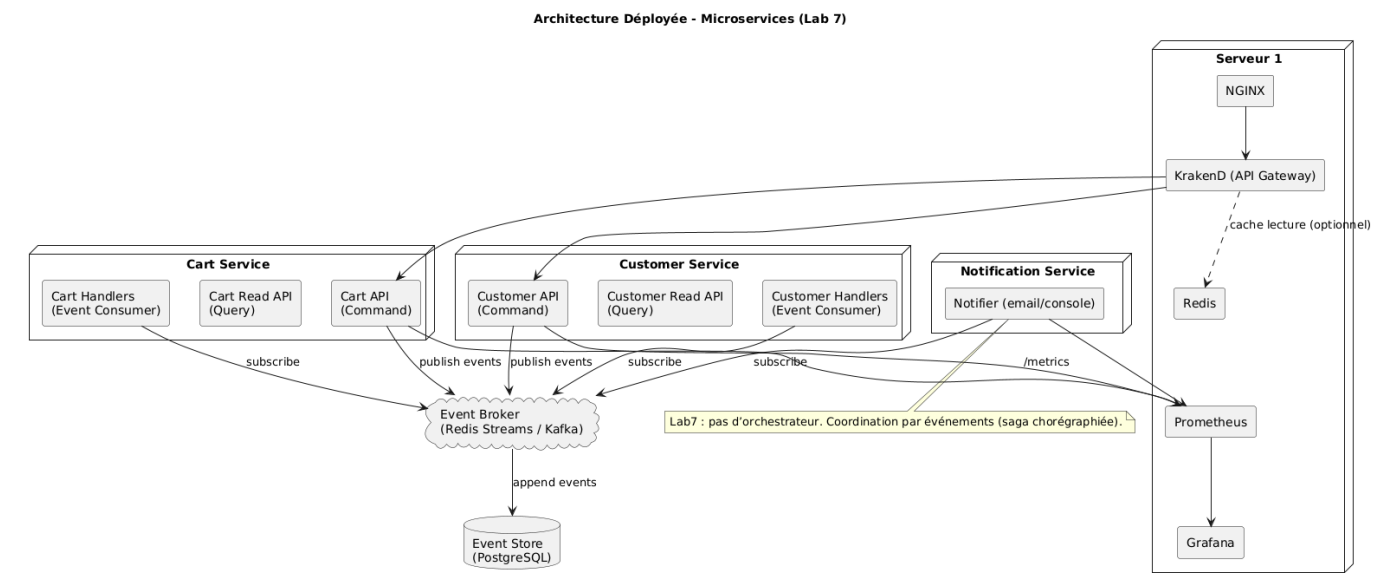


Vue de processus



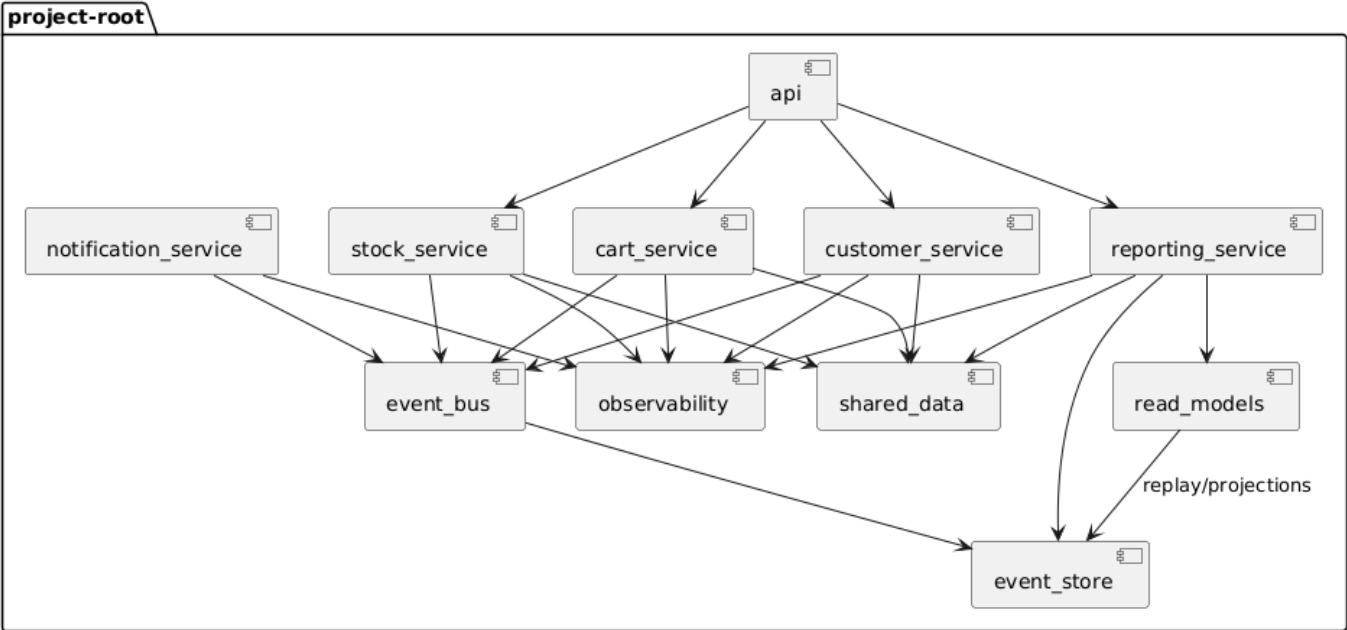
Vue de deploiement

Pas tous les services sont dans le diagramme pour soucis de lisibilité

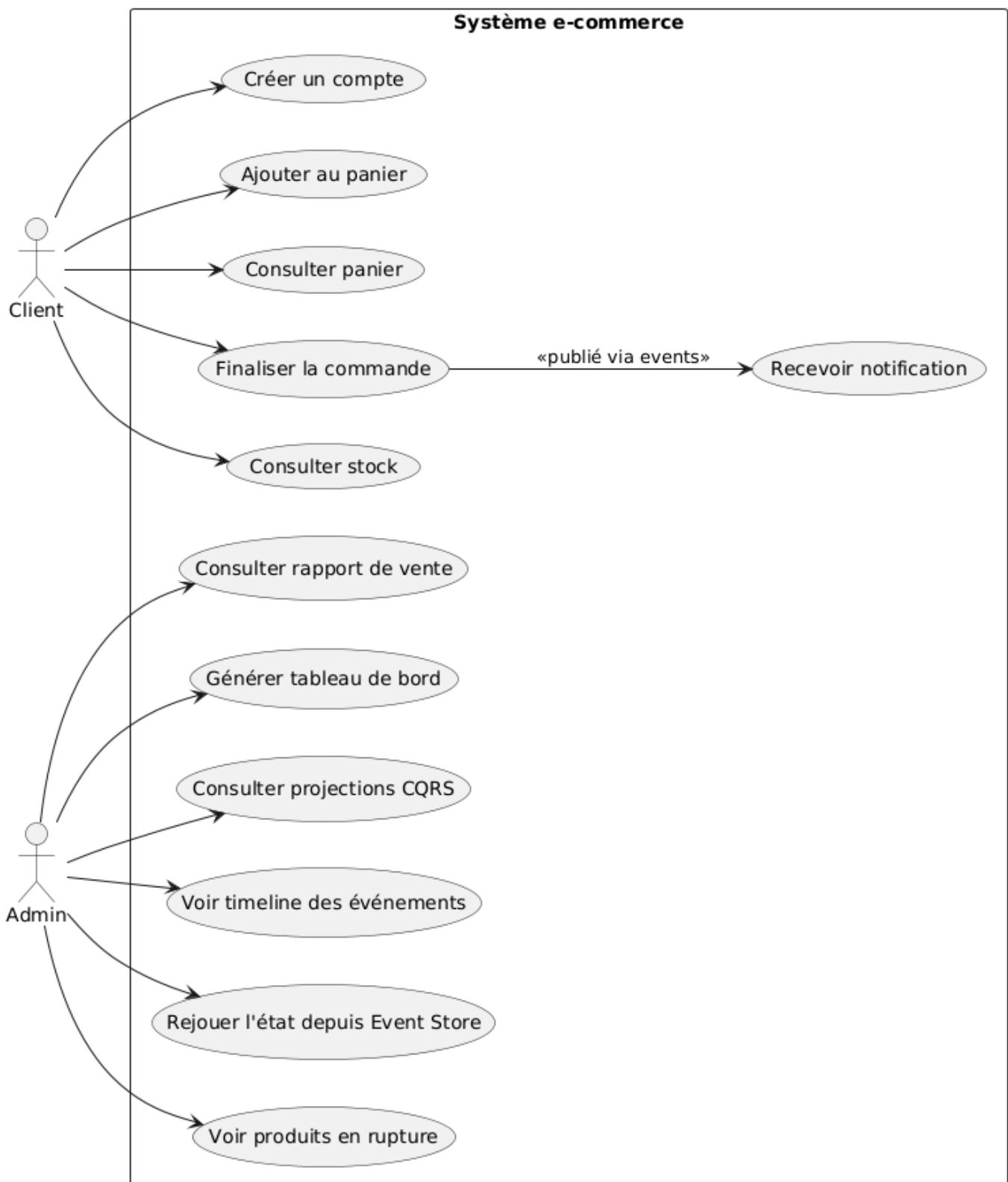


Vue d'implémentation

Project Root (Lab 7)



Vue de cas d'utilisation

Cas d'utilisation - Système e-commerce (Lab 7)

Nouvelles capacités Lab7 :

- UC_CQRS : lectures via "read models"
- UC_Timeline / UC_Replay : Event Sourcing
- UC_Notif : abonné Pub/Sub (notification)

Annexes — gabarits de code (pseudo-Python)

Publisher

```
def publish(event_type: str, payload: dict):
    ev = {
        "eventId": str(uuid4()),
        "type": event_type,
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "payload": payload,
    }
    event_store.append(ev)          # INSERT INTO events ...
    broker.publish(topic_for(event_type), ev) # RabbitMQ/Kafka
```

Consumer

```
def on_message(ev):
    if already_processed(ev["eventId"]):
        return
    try:
        project_or_act(ev)          # update read model, or call domain service
        mark_processed(ev["eventId"])
    except Exception as e:
        dlq(ev, reason=str(e))      # dead-letter if needed
```